Pokhara University

Gandaki College of Engineering and Science

Lamachaur, Pokhara-16, Kaski

Lab 2: Implementation of Source Code Management Using Git Commands Through GitHub – Build, Breakpoints, Refactoring, Collaboration, Copilot, and IntelliSense

Submitted By:

Dev Gurung

Roll no. 20

Submitted To:

Er. Rajendra Bahadur Thapa

Lab Report

Objective

To implement Source Code Management (SCM) using Git and GitHub while utilizing development features such as Build, Breakpoints, Refactoring, Collaboration, GitHub Copilot, and IntelliSense.

Tools Required

- Git
- GitHub
- Visual Studio Code / Visual Studio
- GitHub Copilot (Extension)
- Git Extension
- Sample Project (C++, Java, Python, or C#)

Theory

Source Code Management (SCM) helps developers manage and track changes in source code. Git is a distributed version control system, while GitHub provides a cloud platform for repository hosting and team collaboration. Modern IDEs also provide features like Build, Breakpoints, Refactoring, IntelliSense, and GitHub Copilot to improve software development productivity and code quality.

Procedure

1. Install Git and configure the user details.
2. Create a local Git repository using `git init`.
3. Create a sample project and commit it to the repository.
4. Create a GitHub repository and push the local project.
5. Build the project and verify successful compilation.
6. Set breakpoints and debug the application.
7. Perform code refactoring to improve readability.
8. Use IntelliSense for code completion and error detection.

9. Use GitHub Copilot to generate code suggestions.
10. Collaborate by creating branches, committing changes, and pushing updates to GitHub.

Git Commands (Coding Section)

```
# Configure Git
git config --global user.name "Your Name"
git config --global user.email
"your@email.com"

# Initialize repository
git init

# Add project files
git add .

# Commit changes
git commit -m "Initial project commit"

# Create main branch
git branch -M main

# Connect with GitHub
git remote add origin
https://github.com/username/project.git

# Push to GitHub
git push -u origin main

# Create a new feature branch
git checkout -b feature-update

# Commit branch changes
git add .
git commit -m "Added new feature"
```

```
# Merge branch
git checkout main
git merge feature-update

# View repository status
git status

# View commit history
git log
```

IDE Features Used

Build

Compiled the project successfully and verified that it executed without build errors.

Breakpoints

Used breakpoints during debugging to pause execution, inspect variables, and identify logical errors.

Refactoring

Improved the code structure by renaming variables/functions, removing duplicate code, and enhancing readability without changing functionality.

Collaboration

Managed project versions using Git branches, commits, merges, and synchronization with GitHub for collaborative development.

GitHub Copilot

Used GitHub Copilot to receive AI-powered code suggestions, generate functions, and improve development speed.

IntelliSense

Used IntelliSense for automatic code completion, parameter hints, syntax highlighting, and error detection while coding.

Output

- Git repository initialized successfully.
- Source code uploaded to GitHub.
- Project built successfully.
- Breakpoints used for debugging.
- Code refactored without affecting functionality.
- Collaboration completed using Git branches and GitHub.
- GitHub Copilot assisted with code generation.
- IntelliSense provided code completion and error suggestions.

Result

Successfully implemented Source Code Management using Git commands through GitHub and utilized Build, Breakpoints, Refactoring, Collaboration, GitHub Copilot, and IntelliSense to improve software development and version control.

Conclusion

The experiment demonstrated how Git and GitHub support effective source code management while modern IDE features such as Build, Breakpoints, Refactoring, Collaboration, GitHub Copilot, and IntelliSense enhance code quality, debugging, teamwork, and overall development efficiency.